

PhishGuard Project Blueprint

Explainable & Autonomous Phishing Threat Detection Platform

1. Executive Summary & Problem Statement

The Core Problem: Phishing succeeds because it targets human trust under conditions of manufactured urgency. Existing security solutions provide binary "safe/unsafe" warnings that users routinely ignore or misinterpret. A simple warning label is insufficient when users need understandable evidence before deciding what to do with an urgent message.

The PhishGuard Solution: A layered, explainable AI web application. A user pastes a suspicious email body, URL, or uploads a QR code. PhishGuard doesn't just score the risk—it visually extracts the structural and psychological indicators (impersonation, typosquatting, urgency) into **Evidence Cards**. Crucially, it automatically generates incident reports and network containment rules, turning a passive detection tool into an active defense engine.

2. The "Undeclineable" Factor: Why This Project Wins

Hackathon judges frequently penalize cybersecurity projects that are merely "LLM wrappers" (e.g., passing text to an AI and blindly trusting the output). PhishGuard wins by demonstrating rigorous engineering, deterministic evaluation, and operational SOAR capabilities.

Evaluation Vector	Typical Hackathon Project	The PhishGuard Advantage
Detection Architecture	A single API call asking an LLM "Is this message a scam?"	Deterministic Pipeline: Hardcoded regex/Python logic runs first (subdomains, Punycode). LLM is strictly used for synthesizing the final explanation.
Explainability	Free-form, paragraph-length text generated by an AI model.	Structured Evidence Cards: The UI highlights exact malicious tokens and maps them to verifiable security rules (e.g., Levenshtein distance).
Operational Impact	Displays a red warning box.	Autonomous Response (SOAR): Auto-generates CERT/APWG abuse reports and network blocking rules (DNS Sinkhole/Suricata).
Measurability	Vague claims of accuracy.	Live Benchmarking: Built-in test panel demonstrating precision, recall, and sub-second deterministic evaluation.

3. Multi-Layered Threat Detection Architecture

To prevent AI hallucinations from polluting security telemetry, PhishGuard relies on a staggered inspection pipeline. Input only reaches the Generative layer after it has been rigorously parsed by deterministic Python scripts.

Layer 0: Normalization & Ingestion

Unshortens URLs (expanding bit.ly), decodes QR payloads via OpenCV/Pyzbar, and strips HTML obfuscation. Isolates Top-Level Domains (TLDs) and converts internationalized domains (IDN) to Punycode.



Layer 1: Deterministic Engine (Python/Regex)

Calculates Levenshtein distance against known brand databases (e.g., catching `paypa1.com`). Scans for high-risk TLDs (.xyz, .top) and parses URI open-redirect vulnerabilities.



Layer 2: NLP & Behavioral Extraction

Uses lightweight zero-shot classification or Regex to detect social engineering intent (Urgency, Fear, Greed) and highlights specific token spans for the UI frontend.



Layer 3: Risk Aggregator & Scorer

Computes a composite Risk Score (0-100) based on weighted triggers from Layers 1 & 2. Assigns severity bands: LOW (0-39), MED (40-69), HIGH (70+).



Layer 4: Generative Explainability & Automation (SOAR)

Passes strictly validated JSON telemetry to an LLM to generate plain-English explanations. Triggers automated remediation workflows and abuse report generation.

Technical Stack & Dependencies

Backend (Data & Logic)

- **FastAPI (Python):** High-performance async REST APIs.
- **tlextract & python-Levenshtein:** For URL decomposition and typosquatting detection.
- **pyzbar & opencv-python:** For Quishing (QR code) analysis.
- **LangChain / LiteLLM:** To format the final

Frontend (User Interface)

- **React / Next.js:** Component-based UI structure.
- **Tailwind CSS:** Rapid, clean styling and responsive design.
- **Framer Motion:** For smooth appearance of Evidence Cards.
- **Lucide-React:** Professional iconography for indicators.

4. Security Orchestration, Automation, and Response (SOAR)

What elevates PhishGuard from a simple scanner to an enterprise-grade prototype is its ability to take action. When a high-risk threat (Score ≥ 70) is detected, the system executes an automated containment and reporting workflow.

Automated Abuse Report Generation

PhishGuard automatically formats incident logs tailored for national cybersecurity bodies (e.g., CERT-In, US-CISA, APWG). This removes the friction from reporting cybercrime.

```
===== INCIDENT REPORT:
SUSPECTED PHISHING CAMPAIGN Generated: 2026-09-20T22:15:00Z Platform: PhishGuard Automated
Triage System ===== Target URL:
https://secure-login.wellsfargo.com.account-update.xyz/auth Calculated Risk Index: 96/100
(SEVERITY: HIGH) TECHNICAL EVIDENCE SUMMARY: - [HIGH] Subdomain Deception: Brand 'wellsfargo'
hidden in subdomain. - [HIGH] High-Risk TLD: Domain uses '.xyz', associated with abuse. -
[MED] Coercion: Text contains "suspended within 24 hours". SUBMITTED TO: - Indian Computer
Emergency Response Team (CERT-In) / 1930 Helpline - Anti-Phishing Working Group (APWG)
Repository =====
```

Automated Network Containment Artifacts

Simultaneously, the backend generates indicators of compromise (IOCs) that a security team can immediately deploy to network firewalls and intrusion detection systems.

DNS Sinkhole Format

Blocks the domain at the network level for all corporate users.

```
0.0.0.0 account-update.xyz
```

Suricata / Snort IDS Rule

Alerts the security team if any device attempts to connect.

```
alert http $HOME_NET any -> $EXTERNAL_NET
any (msg:"PHISHGUARD BLOCK";
content:"account-update.xyz"; sid:100001;
rev:1;)
```

Why Judges Care: Hackathons are full of dashboards that just *look* at data. Showing that your system generates real-world defensive artifacts (Suricata rules, CERT templates) proves you understand how cybersecurity operations centers (SOCs) actually work.

5. Hackathon Execution Roadmap (48 Hours)

Follow this phased approach to ensure you have a working MVP well before the submission deadline.

- **Hours 00–12 (Foundation):** Setup FastAPI backend and Next.js frontend skeleton. Implement basic URL decomposition and unshortening utilities. Write the deterministic Levenshtein and TLD rule scripts.
- **Hours 12–24 (Extraction & Scoring):** Integrate QR code parsing (``pyzbar``). Write regex rules for urgency extraction. Establish the scoring matrix algorithm and finalize the JSON schema for backend-frontend communication.
- **Hours 24–36 (SOAR & AI Layer):** Connect the isolated LLM prompt to generate plain-English explanations from the strict JSON evidence. Build the logic to generate the CERT-In abuse reports and DNS block rules.
- **Hours 36–48 (UI Polish & Demo Prep):** Build the interactive "Glass-Box" frontend with sliding Evidence Cards. Hardcode the 3 canonical demo buttons. Rehearse the pitch.

6. The "Glass-Box" Live Demo Strategy

Do not type URLs manually during your presentation. Build three prominent buttons into your UI to instantly load pre-configured scenarios. This guarantees a flawless demo and immediately highlights the system's dynamic explainability.

Scenario	Input Lure	Expected System Output
A: Normal Notice Score: 08/100	"Reminder: The library book return period closes Friday. Visit https://portal.university.edu ."	Result: LOW RISK. Valid <code>`edu`</code> TLD. No coercive deadlines detected. System explains why it is safe.
B: Bank Alert Score: 96/100	"URGENT: Account suspended. Verify immediately at: https://secure.wellsfargo.com.update.xyz/auth "	Result: HIGH RISK. Evidence Cards show: Subdomain impersonation (<code>`update.xyz`</code>), manufactured urgency, credential request. Auto-generates sinkhole rule.
C: Quishing Scam Score: 88/100	(User uploads an image of a QR code offering a "free student grant")	Result: HIGH RISK. Decodes QR payload (<code>`g00gle-scholarships.info`</code>). Identifies character substitution (00 for oo) and plain HTTP protocol.

7. Beyond the Hackathon: Commercial Viability

To win top prizes, you must demonstrate how PhishGuard scales beyond a prototype into a viable product for enterprises and everyday users. Address these points during the Q&A segment of your pitch.

Enterprise Integration Vectors

Slack / MS Teams Bot Integration

Employees can forward suspicious messages directly to a `@PhishGuard` bot in their corporate chat. The bot replies instantly with the Evidence Cards and automatically alerts the IT security desk if the risk is HIGH.

Browser Extension (Consumer)

Package the deterministic engine into a Chrome extension that intercepts DOM rendering. If a user clicks a malicious link from SMS or webmail, the extension halts the redirect and displays the explainable risk overlay.

Target Audience & Go-To-Market Strategy

- B2B (Enterprise Security):** Sold as a lightweight SOAR augmentation. Security Operation Centers (SOCs) spend thousands of hours manually analyzing reported emails. PhishGuard automates the triage, saving enterprise analyst time and reducing alert fatigue.
- B2C (Digital Trust for Vulnerable Populations):** Senior citizens and first-time internet users are disproportionately targeted by financial scams. PhishGuard's "explainable" approach serves as continuous digital literacy training, teaching users *how* to spot scams over time, rather than just blocking them silently.

Impact Metrics (How to measure success in production)

Time-to-Triage (TTT)	Reduces manual email analysis time from ~15 minutes per ticket to under 2 seconds.
False Positive Rate (FPR)	Maintained near 0% by prioritizing deterministic rules (like exact domain mismatch) over easily confused AI interpretations.
User Reporting Uptake	By providing instant, understandable feedback to users who report emails, employee reporting rates increase compared to traditional "black hole" security inboxes.

Final Pitch Closing Statement:

"PhishGuard doesn't just block attacks; it builds human resilience through explainable AI and accelerates enterprise defense through automated orchestration. We are turning the weakest link—the human—into an educated first line of defense."